

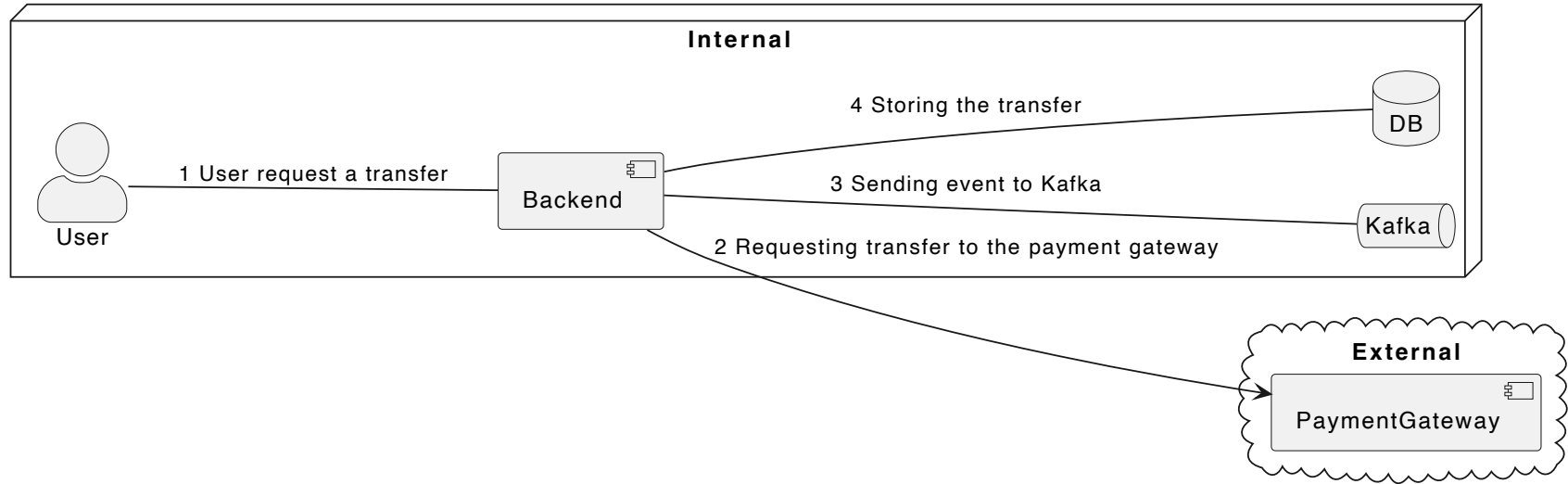


Logging Sucks

(In Go)



Use Case



Who Am I

- Manuel Doncel Martos
- Experience building Microservices



loggingsucks

Who Am I

- Manuel Doncel Martos
- Experience building Microservices



loggingsucks.com

Who Am I

- Manuel Doncel Martos
- Experience building Microservices



loggingsucks.com

Boris Tane



Who Am I

- Manuel Doncel Martos
- Experience building Microservices



loggingsucks.com

Boris Tane



Andrea Agosti

Your logs are misleading you

Table of contents

1. The Why
2. The Fix
3. How (in Go)

The Why

- Many lines for a successful request.
- Most of them saying "nothing" useful.
- And when something goes wrong, they miss context.

The Why

- Many lines for a successful request.
- Most of them saying "nothing" useful.
- And when something goes wrong, they miss context.

The Why

- Many lines for a successful request.
- Most of them saying "nothing" useful.
- And when something goes wrong, they miss context.

The Fix

- Log what happened to this request.
- Logs as structured record of business events.
- For each request, emit one wide event.

The Fix

- Log what happened to this request.
- Logs as structured record of business events.
- For each request, emit one wide event.

The Fix

- Log what happened to this request.
- Logs as structured record of business events.
- For each request, emit one wide event.

How (in Go)

How (in Go)

- Using `context.Context`

How (in Go)

- Using `context.Context`

```
type logEvent struct {  
    mu      sync.RWMutex  
    fields  map[string]any  
}
```


How (in Go)

- Using `context.Context`

```
type logEvent struct {  
    mu      sync.RWMutex  
    fields map[string]any  
}
```

- `logEvent` handled in HTTP middleware

How (in Go)

- Using `context.Context`

```
type logEvent struct {  
    mu      sync.RWMutex  
    fields  map[string]any  
}
```

- `logEvent` handled in HTTP middleware

```
func AddLogEvent(baseLogger *slog.Logger) func(http.Handler) http.Handler {  
    return func (next http.Handler) http.Handler {  
        ...  
        le := &logEvent{  
            fields: make(map[string]any),  
        }  
        le.addField("requestId", reqID)  
        r = r.WithContext(context.WithValue(r.Context(), logEventKey{}, le))  
        next.ServeHTTP(w, r)  
        // log event  
        le.Log()  
        ...  
    }  
}
```

Q/A

